

CSE 207: Data Structures and Algorithms-II (Part 1)

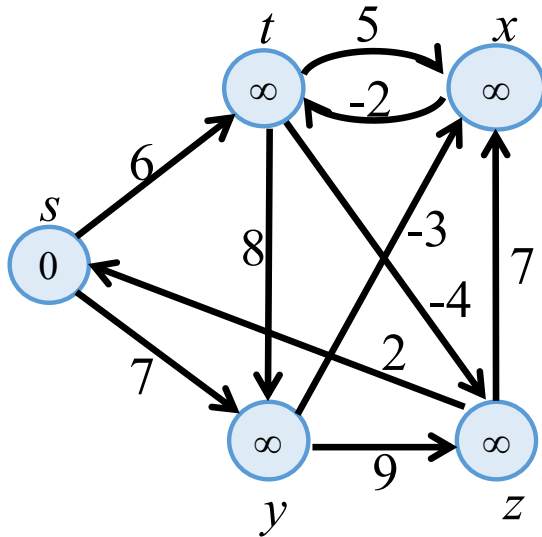
Instructor
Dr Md Monirul Islam

Bellman-Ford Algorithm

BELLMAN-FORD(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$ 
3      for each edge  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
```

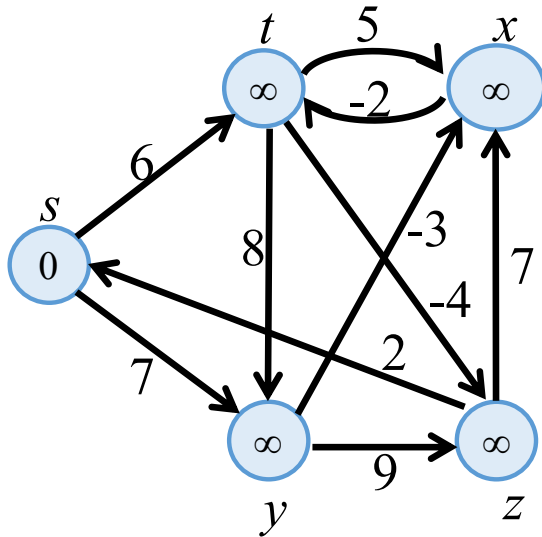
Simulation of Bellman-Ford Algorithm



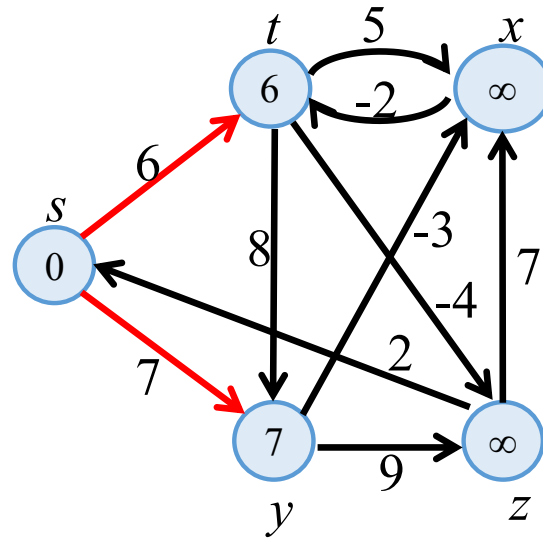
Initialization

We process edges in the order: (t, x) , (t, y) , (t, z) , (x, t) , (y, x) , (y, z) , (z, x) , (z, s) , (s, t) , (s, y)

Simulation of Bellman-Ford Algorithm



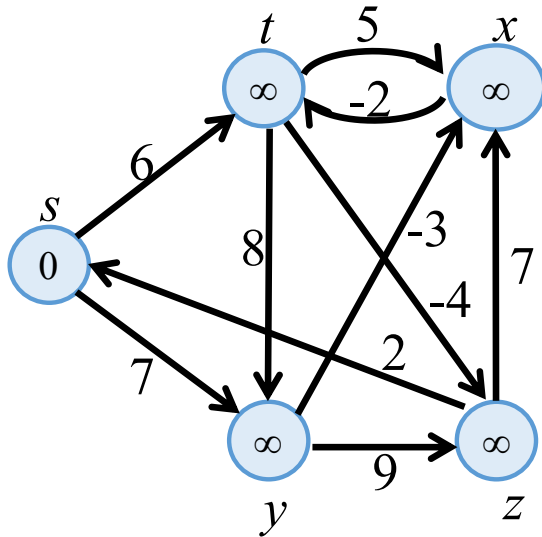
Initialization



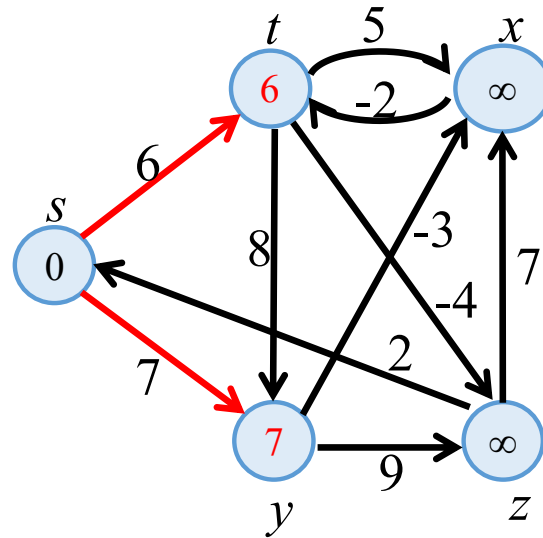
Iteration 1

We process edges in the order: (t, x) , (t, y) , (t, z) , (x, t) , (y, x) , (y, z) , (z, x) , (z, s) , (s, t) , (s, y)

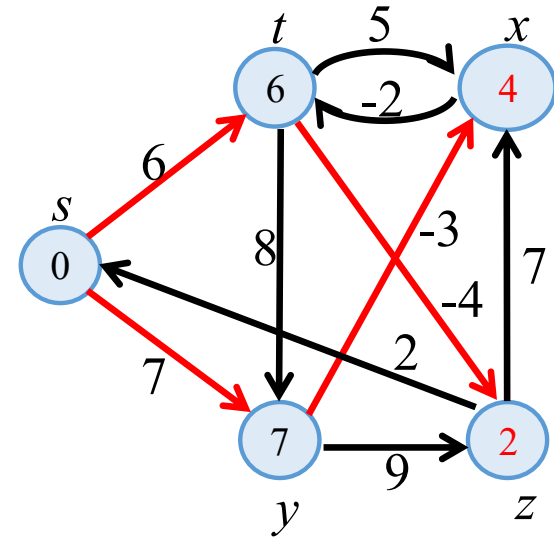
Simulation of Bellman-Ford Algorithm



Initialization



Iteration 1



Iteration 2

How changes:

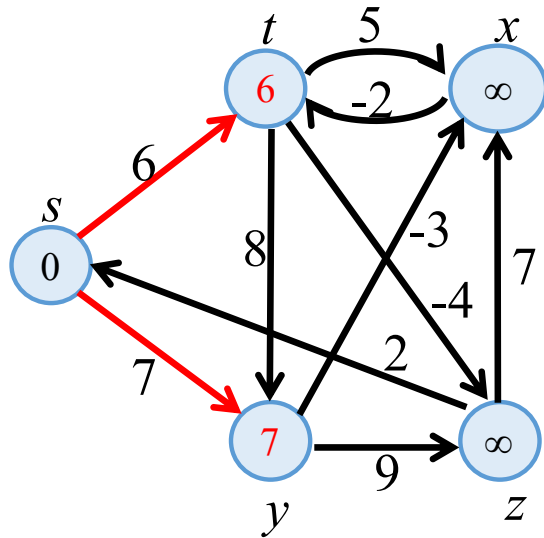
$x=11$

$z=2$

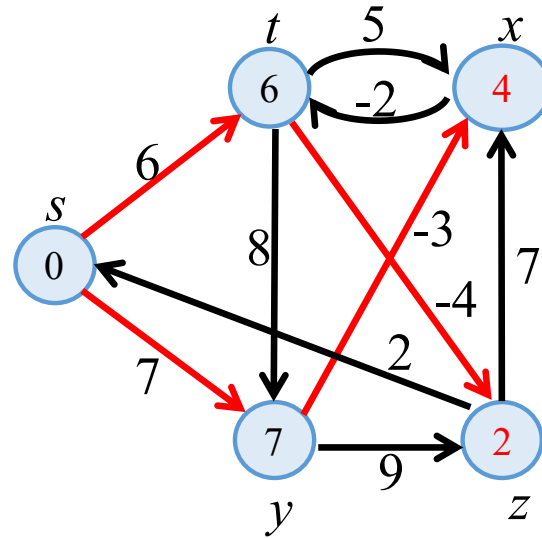
$x=4$

We process edges in the order: (t, x) , (t, y) , (t, z) , (x, t) , (y, x) , (y, z) , (z, x) , (z, s) , (s, t) , (s, y)

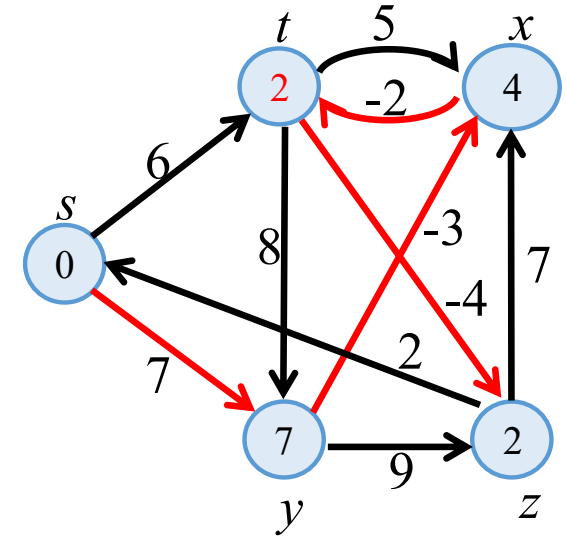
Simulation of Bellman-Ford Algorithm



Iteration 1



Iteration 2



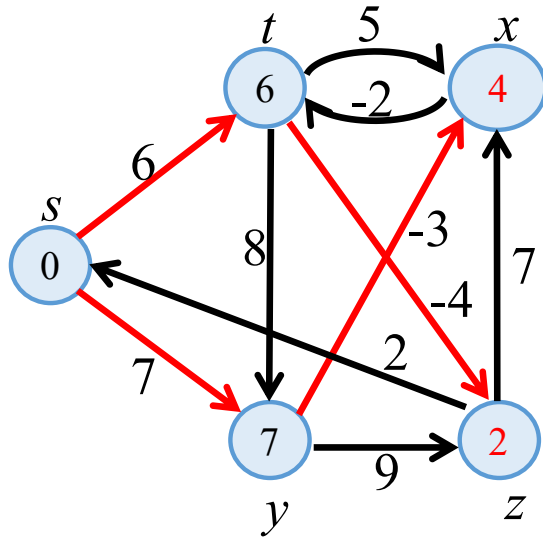
Iteration 3

How changes:

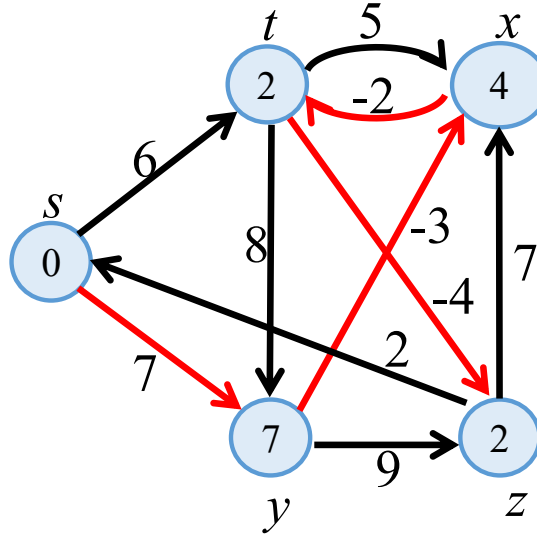
$t=2$

We process edges in the order: (t, x) , (t, y) , (t, z) , (x, t) , (y, x) , (y, z) , (z, x) , (z, s) , (s, t) , (s, y)

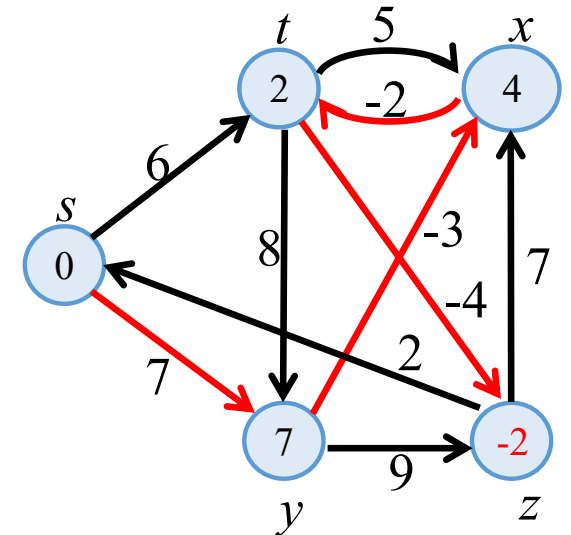
Simulation of Bellman-Ford Algorithm



Iteration 2



Iteration 3



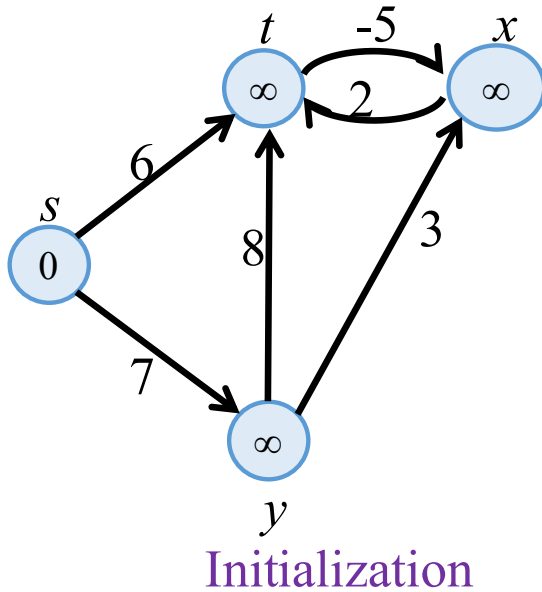
Iteration 4

How changes:

$z = -2$

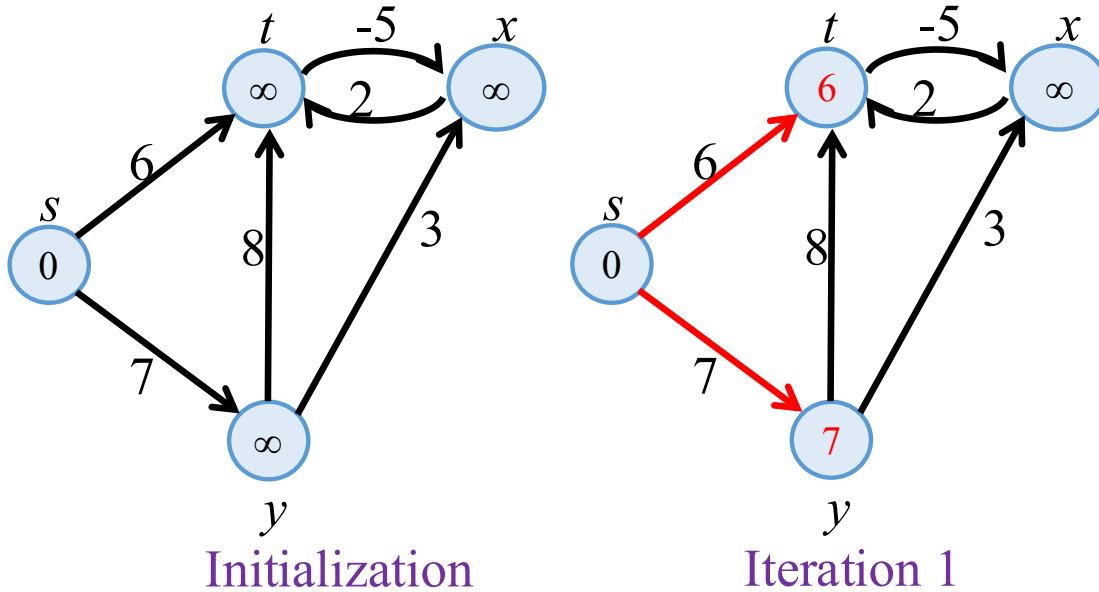
We process edges in the order: (t, x) , (t, y) , (t, z) , (x, t) , (y, x) , (y, z) , (z, x) , (z, s) , (s, t) , (s, y)

Simulation of Bellman-Ford Algorithm: Cycle Detection



We process edges in the order: (t, x) , (x, t) , (y, t) , (y, x) , (s, t) , (s, y)

Simulation of Bellman-Ford Algorithm: Cycle Detection

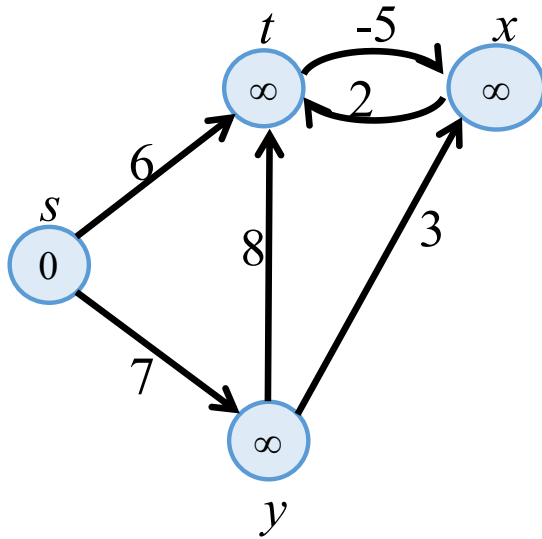


How changes:

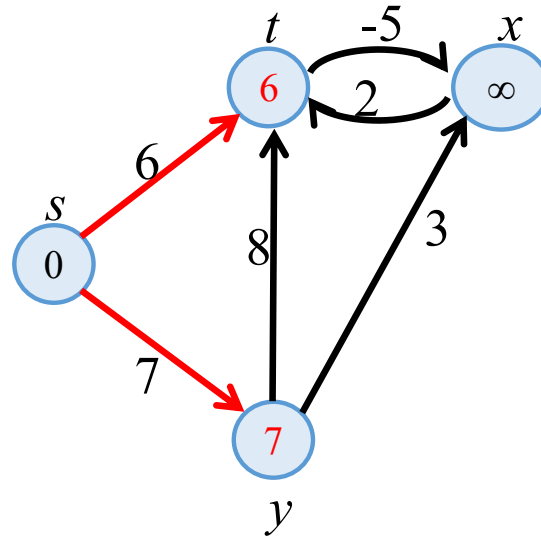
$t=6$ $y=7$

We process edges in the order: (t, x) , (x, t) , (y, t) , (y, x) , (s, t) , (s, y)

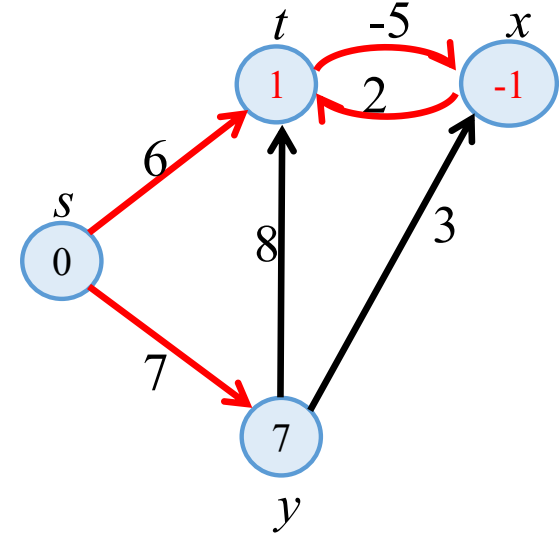
Simulation of Bellman-Ford Algorithm: Cycle Detection



Initialization



Iteration 1



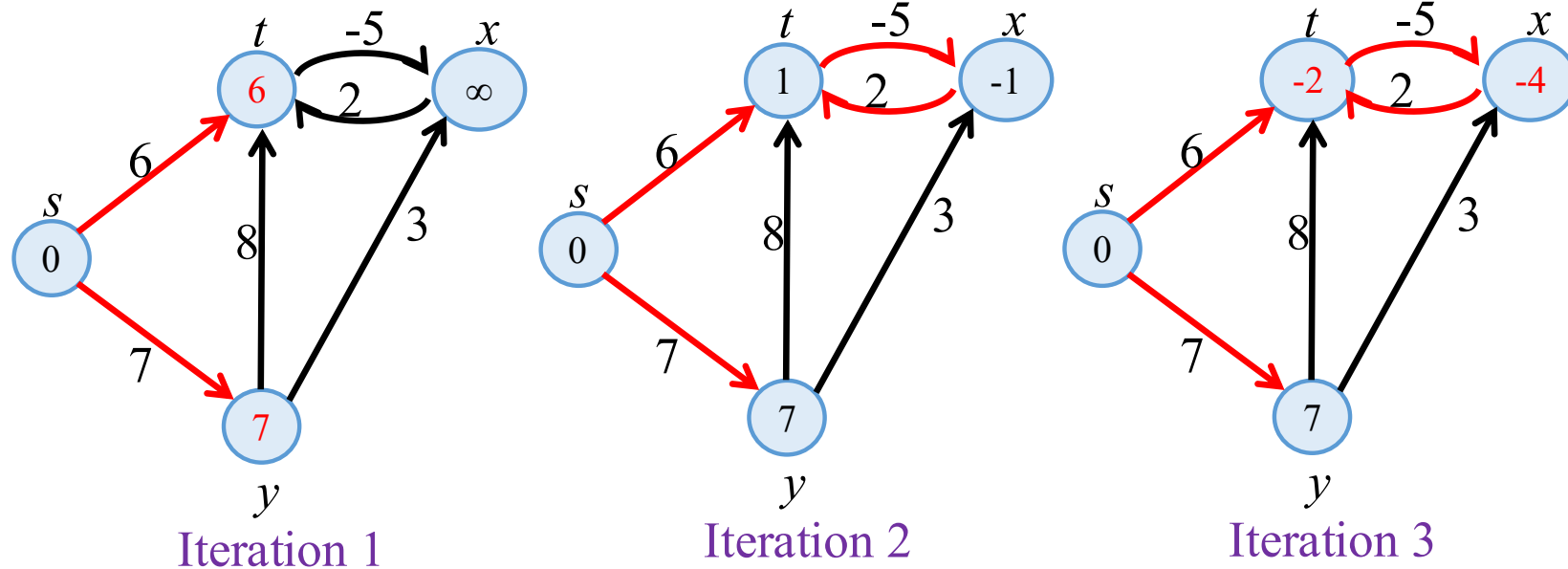
Iteration 2

How changes:

$x=-1$ $t=1$

We process edges in the order: (t, x) , (x, t) , (y, t) , (y, x) , (s, t) , (s, y)

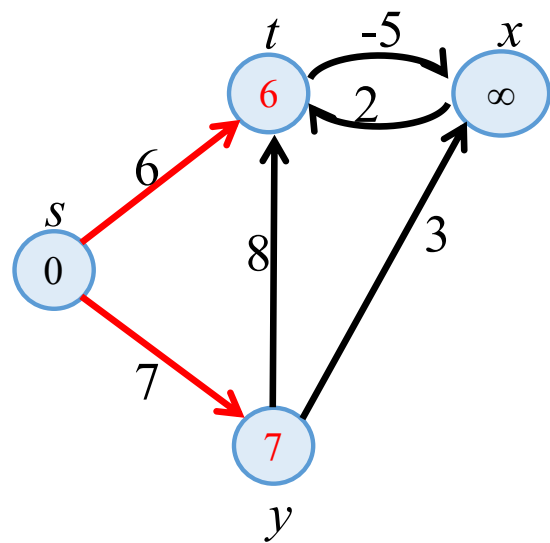
Simulation of Bellman-Ford Algorithm: Cycle Detection



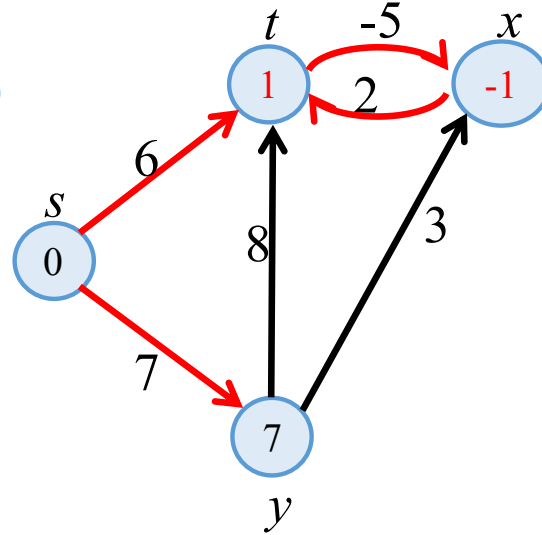
How changes: $x=-4$ $t=-2$

We process edges in the order: (t, x) , (x, t) , (y, t) , (y, x) , (s, t) , (s, y)

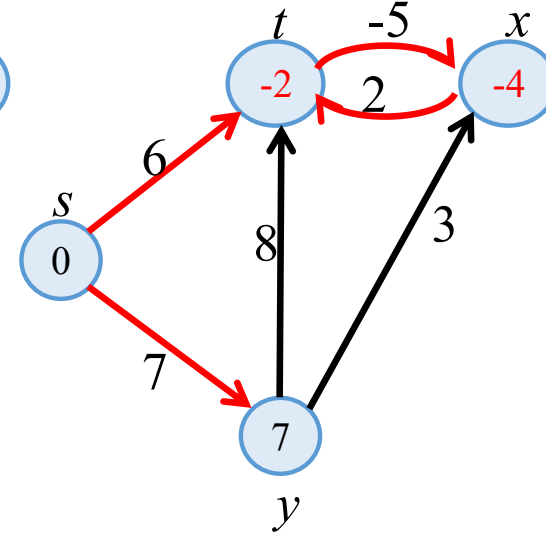
Simulation of Bellman-Ford Algorithm: Cycle Detection



Iteration 1



Iteration 2



Iteration 3

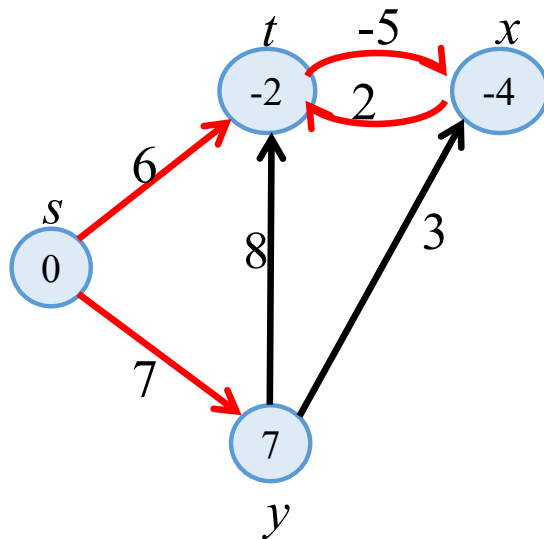
Main iterations are completed!

How changes:

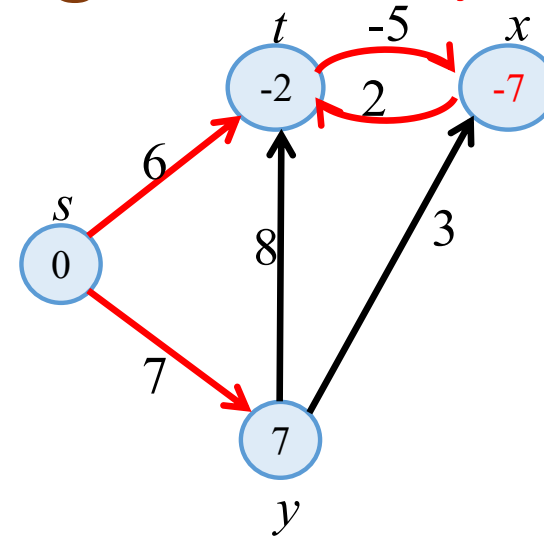
$x=-4$ $t=-2$

We process edges in the order: (t, x) , (x, t) , (y, t) , (y, x) , (s, t) , (s, y)

Simulation of Bellman-Ford Algorithm: Cycle Detection



Iteration 3



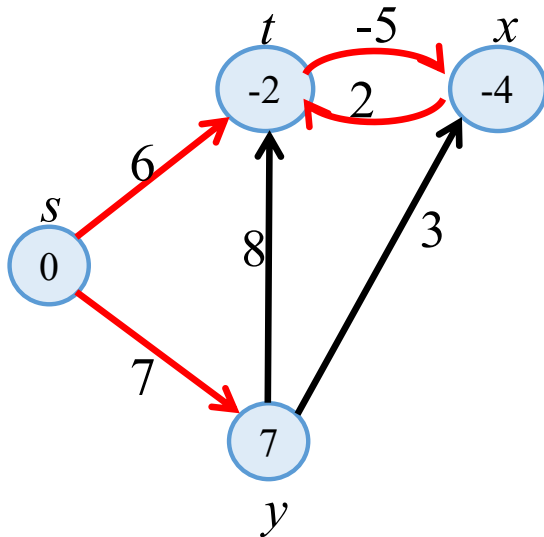
Extra Iteration for
Cycle Detection

Cycle Detection
iteration

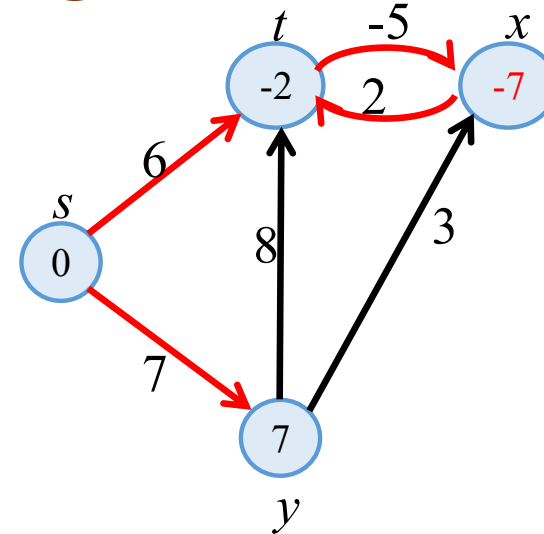
How changes: $x = -7$

We process edges in the order: (t, x) , (x, t) , (y, t) , (y, x) , (s, t) , (s, y)

Simulation of Bellman-Ford Algorithm: Cycle Detection



Iteration 3



Extra Iteration for
Cycle Detection

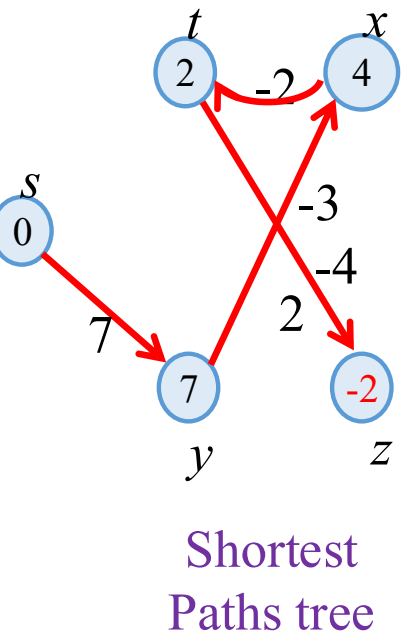
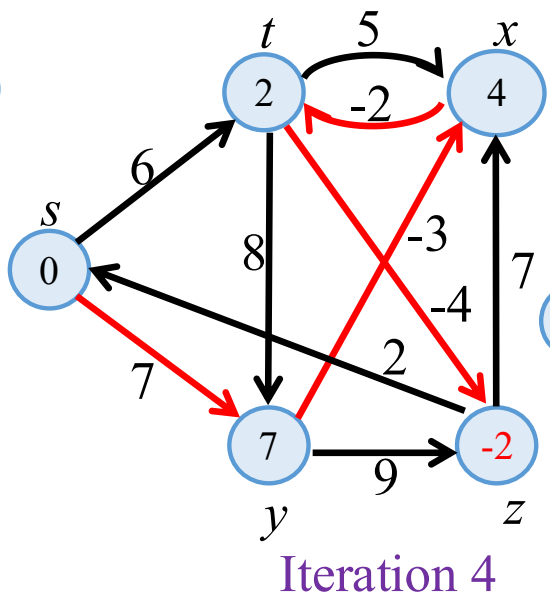
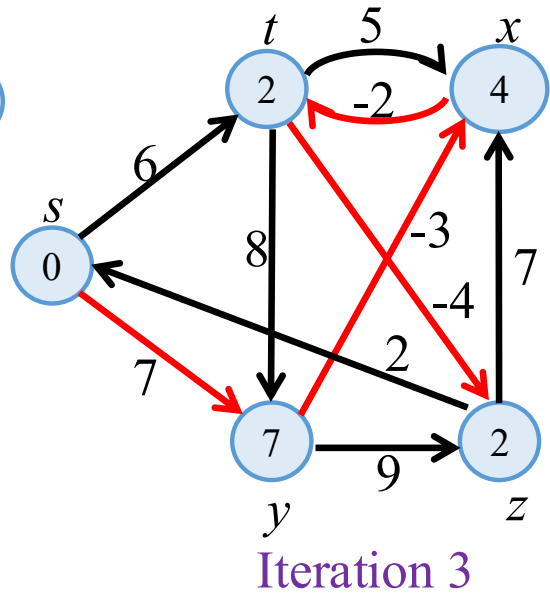
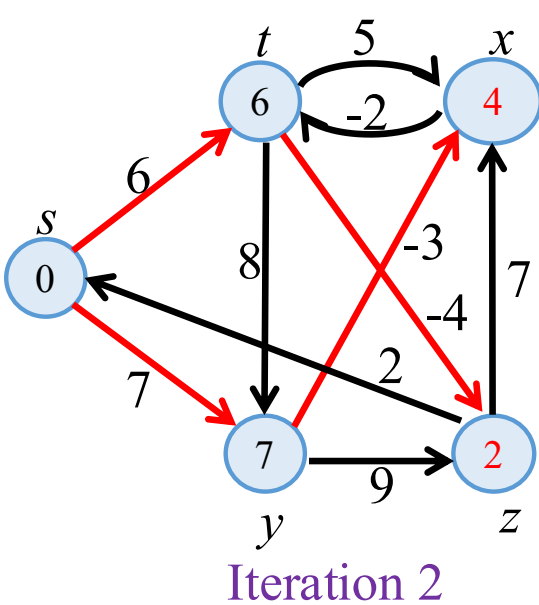
Detected!!

How changes:

$x = -7$

We process edges in the order: (t, x) , (x, t) , (y, t) , (y, x) , (s, t) , (s, y)

Bellman-Ford Algorithm Correctly Finds Shortest Paths



Bellman-Ford Algorithm Correctly Finds Shortest Paths

Lemma 24.2

After first $|V|-1$ relaxation iterations of the algorithm on a weighted, directed graph $G = (V, E)$ with no negative-weight cycle reachable from s , we have $v.d = \delta(s, v)$ for all vertices v reachable from s .

Bellman-Ford Algorithm Correctly Finds Shortest Paths

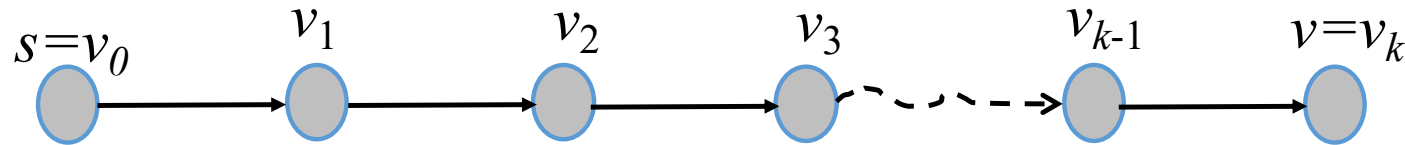
Lemma 24.2

After first $|V|-1$ relaxation iterations of the algorithm on a weighted, directed graph $G = (V, E)$ with no negative-weight cycle, we have $v.d = \delta(s, v)$ for all vertices v reachable from s .

Proof of Lemma 24.2

$p = \langle v_0, v_1, \dots, v_k \rangle$ is a shortest path from $s = v_0$ to $v = v_k$

p is a simple path $\Rightarrow p$ has at most $|V|-1$ edges



$$k \leq |V| - 1$$

Bellman-Ford Algorithm Correctly Finds Shortest Paths

Lemma 24.2

After first $|V|-1$ relaxation iterations of the algorithm on a weighted, directed graph $G = (V, E)$ with no negative-weight cycle, we have $v.d = \delta(s, v)$ for all vertices v reachable from s .

Proof of Lemma 24.2

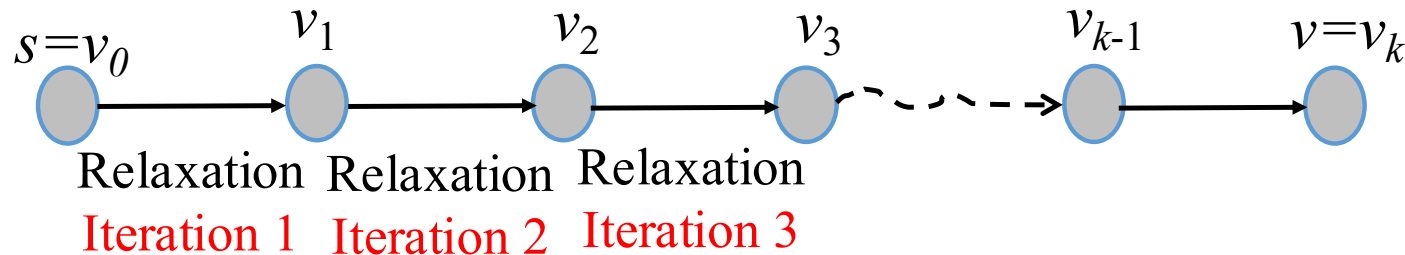
The algorithm relaxes all edges $|V|-1$ times

Therefore, edge (v_0, v_1) is relaxed at iteration 1

edge (v_1, v_2) is relaxed at iteration 2

edge (v_2, v_3) is relaxed at iteration 3

$$k \leq |V| - 1$$



Bellman-Ford Algorithm Correctly Finds Shortest Paths

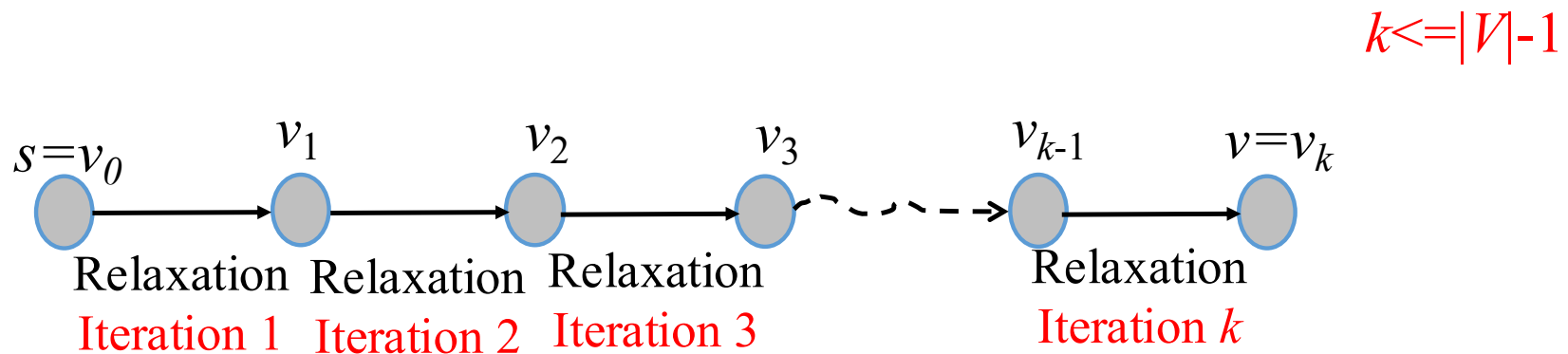
Lemma 24.2

After first $|V|-1$ relaxation iterations of the algorithm on a weighted, directed graph $G = (V, E)$ with no negative-weight cycle, we have $v.d = \delta(s, v)$ for all vertices v reachable from s .

Proof of Lemma 24.2

The algorithm relaxes all edges $|V|-1$ times

Therefore, edge (v_{i-1}, v_i) is relaxed at iteration i ; for $i = 1$ to k



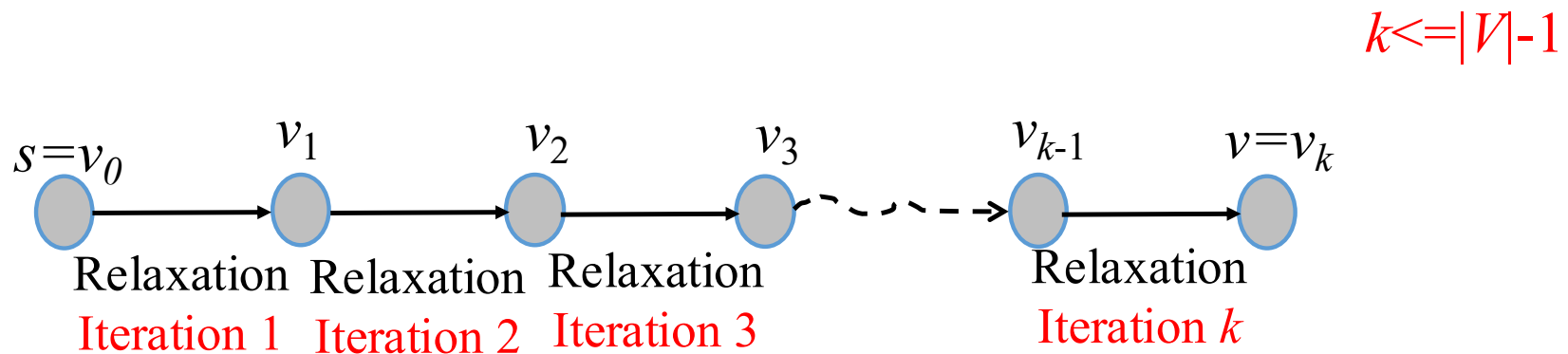
Bellman-Ford Algorithm Correctly Finds Shortest Paths

Lemma 24.2

After first $|V|-1$ relaxation iterations of the algorithm on a weighted, directed graph $G = (V, E)$ with no negative-weight cycle, we have $v.d = \delta(s, v)$ for all vertices v reachable from s .

Proof of Lemma 24.2

Path relaxation property says that $v.d = v_k.d = \delta(s, v_k) = \delta(s, v)$



Bellman-Ford Algorithm Correctly Finds Shortest Paths

Corollary 24.3

Let $G = (V, E)$ be a weighted, directed graph with no negative-weight cycle reachable from s , we have $v.d = \delta(s, v)$ for all vertices v reachable from s .

Then, for each vertex v , there is a path from s to v if and only if BELLMAN-FORD terminates with $v.d < \infty$ when it is run on G .

Bellman-Ford Algorithm Correctly Finds Shortest Paths

Corollary 24.3

Let $G = (V, E)$ be a weighted, directed graph with no negative-weight cycle reachable from s , we have $v.d = \delta(s, v)$ for all vertices v reachable from s .

Then, for each vertex v , there is a path from s to v if and only if BELLMAN-FORD terminates with $v.d < \infty$ when it is run on G .

We can shorten the statement as follows

P if and only if Q

where,

P = there is a path from s to v

Q = BELLMAN-FORD terminates with $v.d < \infty$ when it is run on G

Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of Corollary 24.3

If P, then Q

where,

P = there is a path from s to v

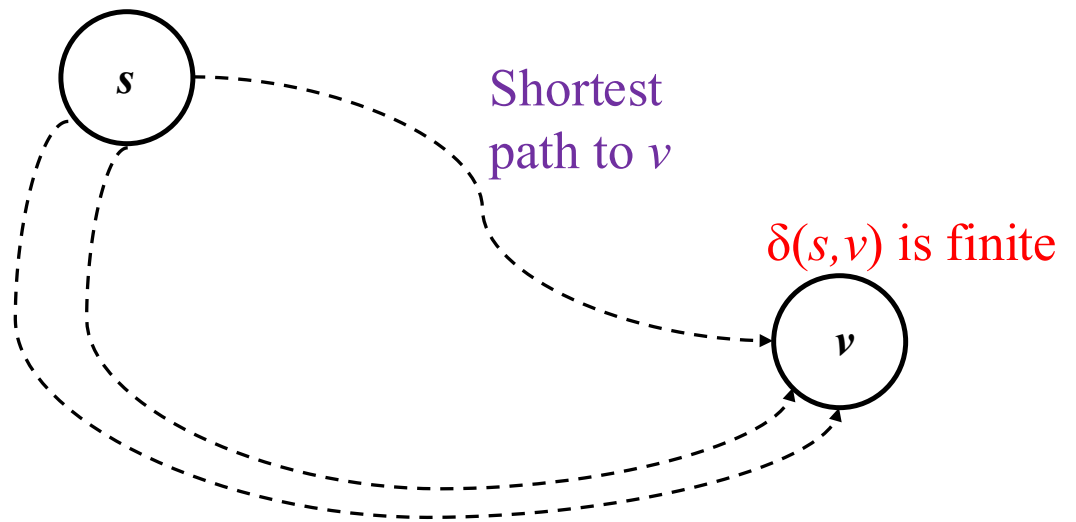
Q = BELLMAN-FORD terminates with $v.d < \infty$ when it is run on G

Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of Corollary 24.3

If P, then Q

As there is a path from s to v , *there is a shortest too.* $\Rightarrow \delta(s, v)$ is finite



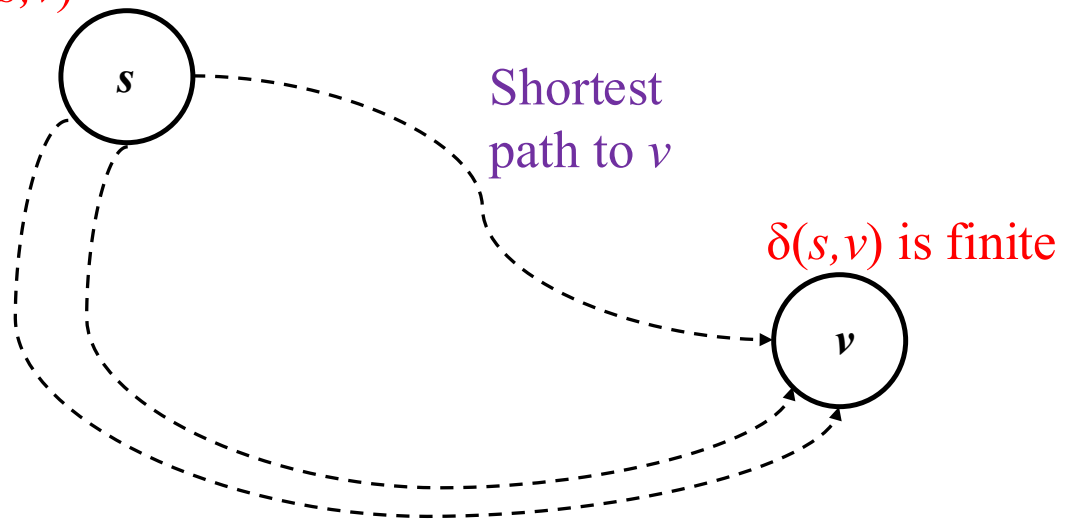
Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of Corollary 24.3

If P, then Q

As there is a path from s to v , *there is a shortest too.* $\Rightarrow \delta(s, v)$ is finite

Lemma 24.2 tells that line 2-4 sets $v.d = \delta(s, v) < \infty$



Bellman-Ford Algorithm Correctly Finds Shortest Paths

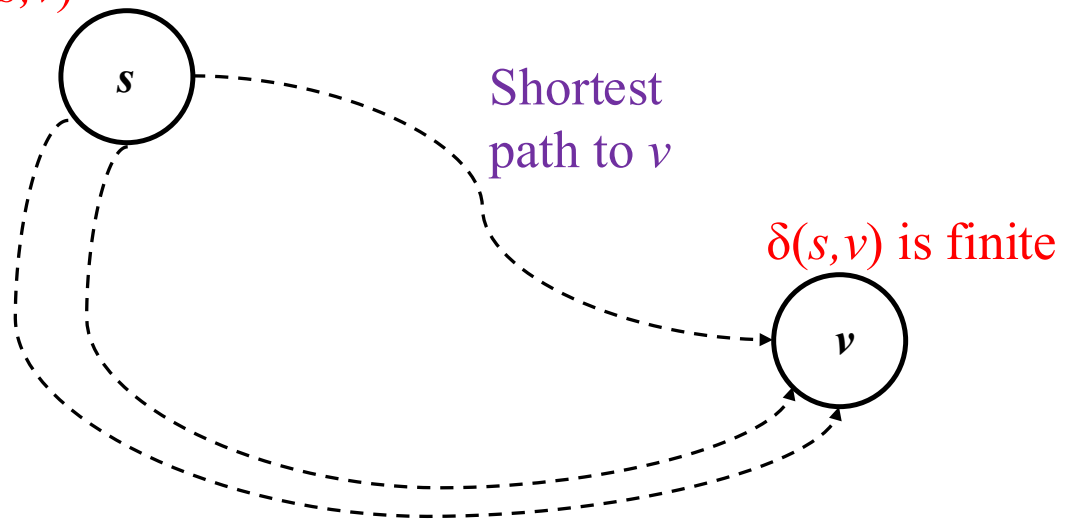
Proof of Corollary 24.3

If P, then Q

As there is a path from s to v , *there is a shortest too.* $\Rightarrow \delta(s, v)$ is finite

Lemma 24.2 tells that line 2-4 sets $v.d = \delta(s, v) < \infty$

Therefore, $v.d < \infty$



Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of Corollary 24.3

If Q, then P

where,

P = there is a path from s to v

Q = BELLMAN-FORD terminates with $v.d < \infty$ when it is run on G

Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of Corollary 24.3

If Q, then P

Initially all $v.d = \infty$

We gradually decrease $v.d$ and set $v.\pi$ by relaxation if there is a path from s to v

BELLMAN-FORD(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$ 
3      for each edge  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
```

Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of Corollary 24.3

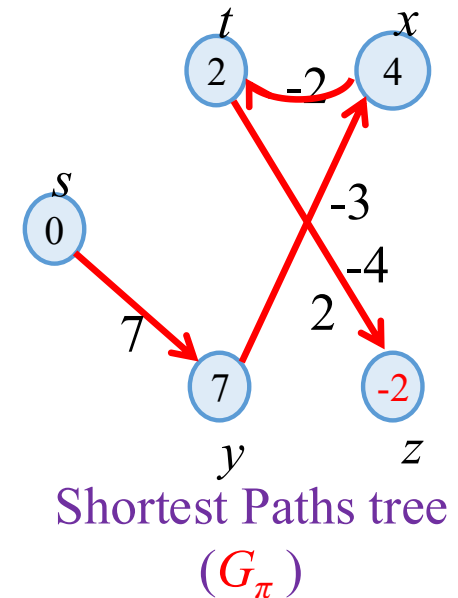
If Q, then P

Initially all $v.d = \infty$

We gradually **decrease $v.d$** and **set $v.\pi$** by relaxation if there is a path from s to v

G_π forms and it is *shortest path tree rooted at s* .

Therefore, there is a **path from s to v** in G_π



Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of Corollary 24.3

If Q, then P

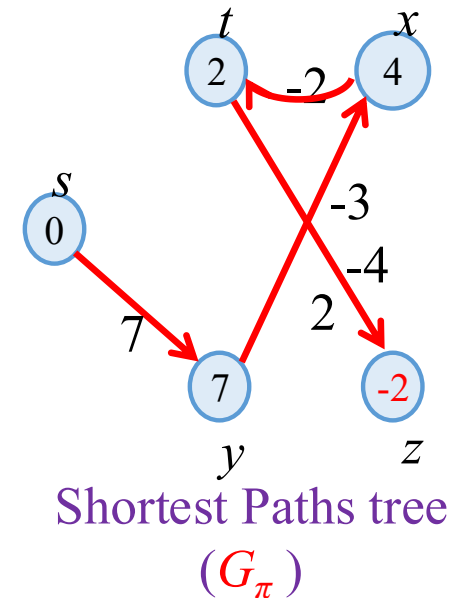
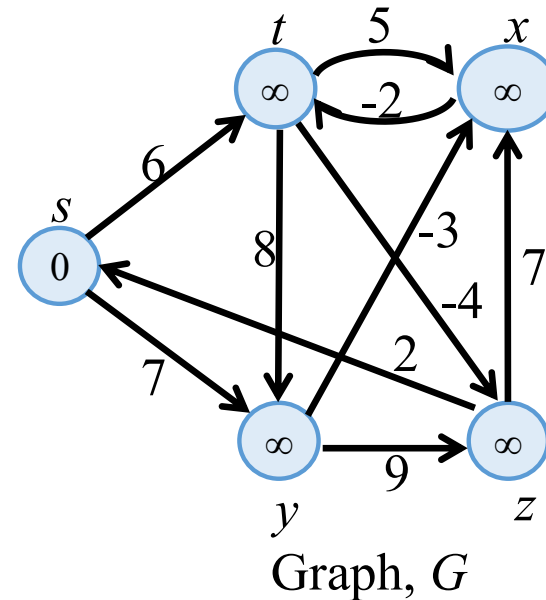
Initially all $v.d = \infty$

We gradually **decrease $v.d$** and **set $v.\pi$** by relaxation if there is a path from s to v

G_π forms and it is *shortest path tree rooted at s* .
Therefore, there is a **path from s to v** in G_π

Edges of this path are also in G

Therefore, there is a **path from s to v** in G also.



Bellman-Ford Algorithm Correctly Finds Shortest Paths

Theorem 24.4

Let $G = (V, E)$ be a weighted, directed graph.

(A) If G has no negative-weight cycle reachable from s

Then, (i) the algorithm returns true (ii) $v.d = \delta(s, v)$, and (iii) G_π is a shortest path tree rooted as s .

(B) If G has negative-weight cycle reachable from s , the algorithm returns false

Bellman-Ford Algorithm Correctly Finds Shortest Paths

Theorem 24.4

Let $G = (V, E)$ be a weighted, directed graph.

(A) If G has no negative-weight cycle reachable from s

Then, (i) the algorithm returns true (ii) $v.d = \delta(s, v)$, and (iii) G_π is a shortest path tree rooted at s .

Proof of A

(ii) From Lemma 24.2, $v.d = \delta(s, v) < \infty$ for all reachable vertex v from s

If v is not reachable. $v.d = \infty = \delta(s, v)$

Therefore, A(ii) is proved.

Bellman-Ford Algorithm Correctly Finds Shortest Paths

Theorem 24.4

Let $G = (V, E)$ be a weighted, directed graph.

(A) If G has no negative-weight cycle reachable from s

Then, (i) the algorithm returns true (ii) $v.d = \delta(s, v)$, and (iii) G_π is a shortest path tree rooted at s .

Proof of A

(iii) $v.d = \delta(s, v)$ for all v . **By (ii)**

Bellman-Ford Algorithm Correctly Finds Shortest Paths

Theorem 24.4

Let $G = (V, E)$ be a weighted, directed graph.

(A) If G has no negative-weight cycle reachable from s

Then, (i) the algorithm returns true (ii) $v.d = \delta(s, v)$, and (iii) G_π is a shortest path tree rooted as s .

Proof of A

(iii) $v.d = \delta(s, v)$ for all v .

the predecessor subgraph property says

if $v.d = \delta(s, v)$ for all v then G_π is a shortest path tree rooted as s

Bellman-Ford Algorithm Correctly Finds Shortest Paths

Theorem 24.4

Let $G = (V, E)$ be a weighted, directed graph.

(A) If G has no negative-weight cycle reachable from s

Then, (i) the algorithm returns true (ii) $v.d = \delta(s, v)$, and (iii) G_π is a shortest path tree rooted at s .

Proof of A

(i) Statement A(ii) says,

$v.d = \delta(s, v)$

Bellman-Ford Algorithm Correctly Finds Shortest Paths

Theorem 24.4

Let $G = (V, E)$ be a weighted, directed graph.

(A) If G has no negative-weight cycle reachable from s

Then, (i) the algorithm returns true (ii) $v.d = \delta(s, v)$, and (iii) G_π is a shortest path tree rooted at s .

Proof of A

(i) Statement A(ii) says,

$$v.d = \delta(s, v)$$

$$\leq \delta(s, u) + w(u, v) \text{ by triangular inequality}$$

$$\leq u.d + w(u, v) \quad \text{by Statement A(ii) } u.d = \delta(s, u)$$

Bellman-Ford Algorithm Correctly Finds Shortest Paths

Theorem 24.4

Let $G = (V, E)$ be a weighted, directed graph.

(A) If G has no negative-weight cycle reachable from s

Then, (i) the algorithm returns true (ii) $v.d = \delta(s, v)$, and (iii) G_π is a shortest path tree rooted at s .

Proof of A

(i) Statement A(ii) says,

$$v.d = \delta(s, v)$$

$$\leq \delta(s, u) + w(u, v) \text{ by triangular inequality}$$

$$\leq u.d + w(u, v) \quad \text{by Statement A(ii) } u.d = \delta(s, u)$$

Therefore, $v.d \leq u.d + w(u, v)$ for every edge (u, v)

Bellman-Ford Algorithm Correctly Finds Shortest Paths

Theorem 24.4

Let $G = (V, E)$ be a weighted, directed graph.

(A) If G has no negative-weight cycle reachable from s

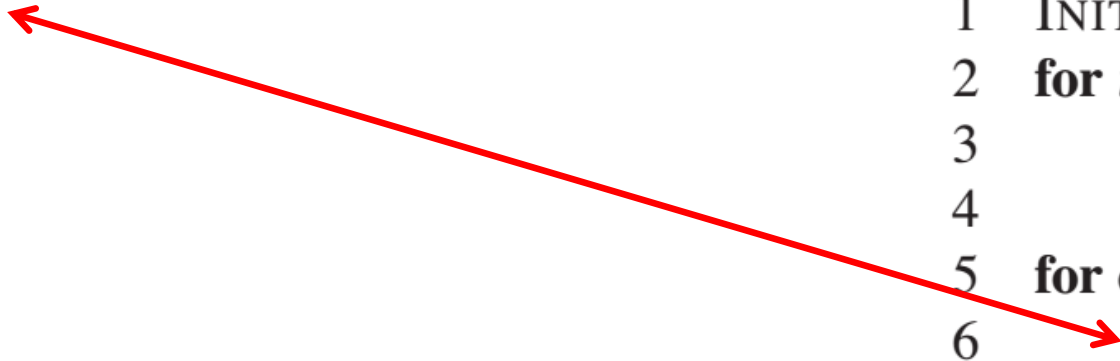
Then, (i) the algorithm returns true (ii) $v.d = \delta(s, v)$, and (iii) G_π is a shortest path tree rooted as s .

Proof of A

(i) $v.d \leq u.d + w(u, v)$ for every edge (u, v)

BELLMAN-FORD(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$ 
3      for each edge  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
```



Bellman-Ford Algorithm Correctly Finds Shortest Paths

Theorem 24.4

Let $G = (V, E)$ be a weighted, directed graph.

(A) If G has no negative-weight cycle reachable from s

Then, (i) the algorithm returns true (ii) $v.d = \delta(s, v)$, and (iii) G_π is a shortest path tree rooted at s .

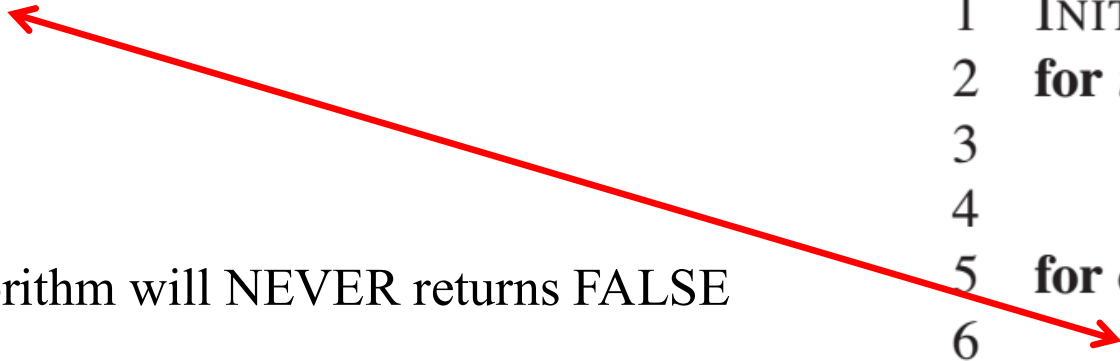
Proof of A

(i) $v.d \leq u.d + w(u, v)$ for every edge (u, v)

The algorithm will NEVER return FALSE

BELLMAN-FORD(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$ 
3      for each edge  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
```



Bellman-Ford Algorithm Correctly Finds Shortest Paths

Theorem 24.4

Let $G = (V, E)$ be a weighted, directed graph.

(A) If G has no negative-weight cycle reachable from s

Then, (i) the algorithm returns true (ii) $v.d = \delta(s, v)$, and (iii) G_π is a shortest path tree rooted as s .

(B) If G has negative-weight cycle reachable from s , the algorithm returns false

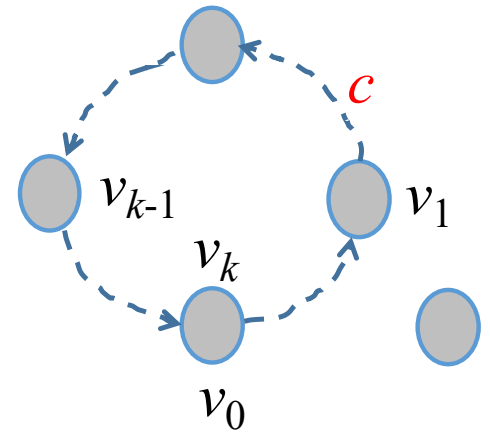
Proof of B:

Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of B:

Assume we have negative weight cycle, $c = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = v_k$

As c is a negative weight cycle, $\sum_{i=1}^k w(v_{i-1}, v_i) < 0$



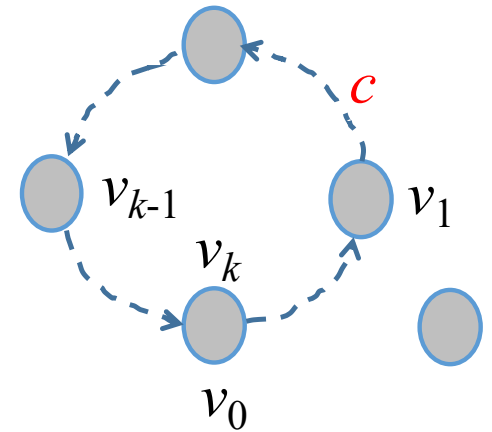
Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of B:

Assume for the contradiction, algorithm returns TRUE

BELLMAN-FORD(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$ 
3      for each edge  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
```



Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of B:

Assume for the contradiction, algorithm returns TRUE

BELLMAN-FORD(G, w, s)

1 INITIALIZE-SINGLE-SOURCE(G, s)

2 **for** $i = 1$ **to** $|G.V| - 1$

3 **for** each edge $(u, v) \in G.E$

4 RELAX(u, v, w)

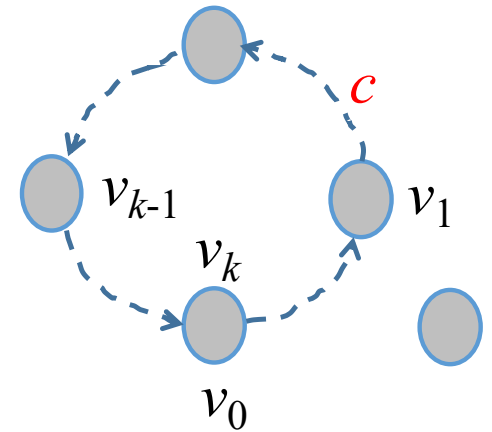
5 **for** each edge $(u, v) \in G.E$

6 **if** $v.d > u.d + w(u, v)$

7 **return** FALSE

8 **return** TRUE

This will never execute



Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of B:

Assume for the contradiction, algorithm returns TRUE

BELLMAN-FORD(G, w, s)

1 INITIALIZE-SINGLE-SOURCE(G, s)

2 **for** $i = 1$ **to** $|G.V| - 1$

3 **for** each edge $(u, v) \in G.E$

4 RELAX(u, v, w)

5 **for** each edge $(u, v) \in G.E$

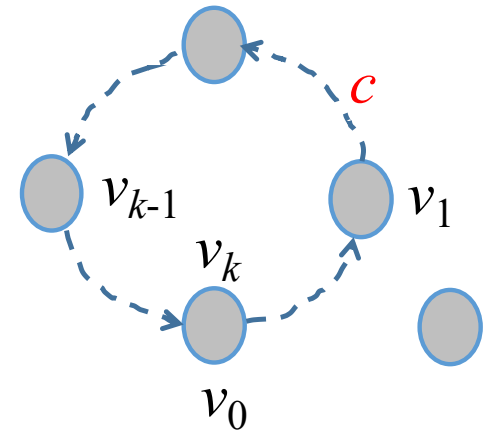
6 **if** $v.d > u.d + w(u, v)$

7 **return** FALSE

8 **return** TRUE

This is false always

This will never execute



Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of B:

Assume for the contradiction, algorithm returns TRUE

BELLMAN-FORD(G, w, s)

1 INITIALIZE-SINGLE-SOURCE(G, s)

2 **for** $i = 1$ **to** $|G.V| - 1$

3 **for** each edge $(u, v) \in G.E$

4 RELAX(u, v, w)

5 **for** each edge $(u, v) \in G.E$

6 **if** $v.d > u.d + w(u, v)$

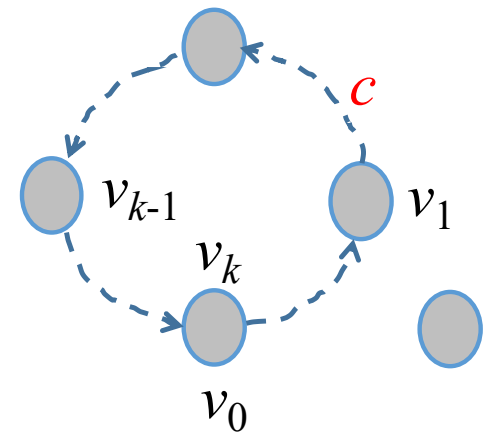
7 **return** FALSE

8 **return** TRUE

This means,
 $v.d \leq u.d + w(u, v)$
for all edges in c

This is false always

This will never execute



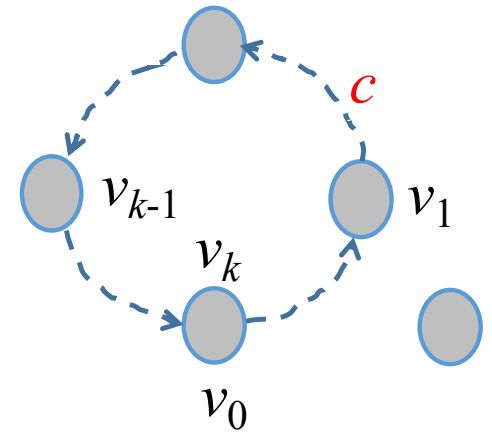
Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of B:

Assume for the contradiction, algorithm returns TRUE

$$v_i.d \leq v_{i-1}.d + w(v_{i-1}, v_i) \text{ for all } i = 0 \text{ to } k$$

This means,
 $v.d \leq u.d + w(u, v)$
for all edges in c



Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of B:

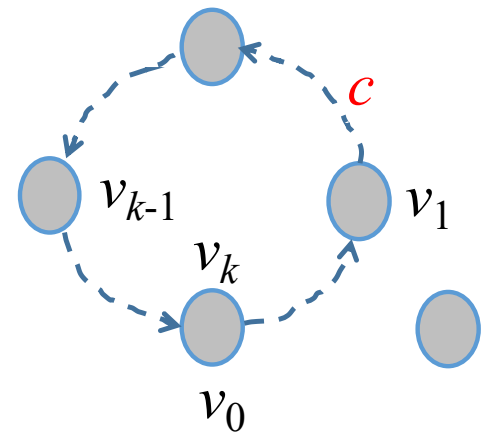
Assume for the contradiction, algorithm returns TRUE

$$v_1.d \leq v_0.d + w(v_0, v_1) \text{ for } i=0$$

$$v_2.d \leq v_1.d + w(v_1, v_2) \text{ for } i=1$$

$$v_3.d \leq v_2.d + w(v_2, v_3) \text{ for } i=2$$

$$v_k.d \leq v_{k-1}.d + w(v_{k-1}, v_k) \text{ for } i=k$$



Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of B:

Assume for the contradiction, algorithm returns TRUE

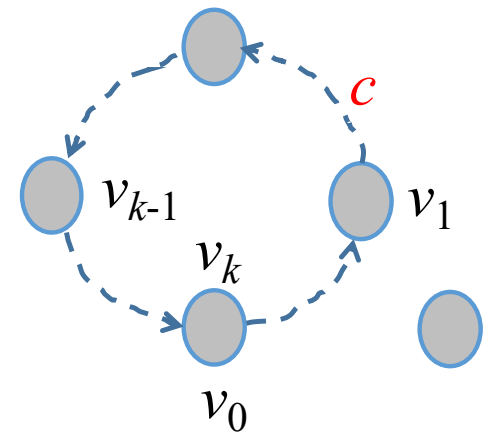
$$v_1.d \leq v_0.d + w(v_0, v_1) \text{ for } i=0$$

$$v_2.d \leq v_1.d + w(v_1, v_2) \text{ for } i=1$$

$$v_3.d \leq v_2.d + w(v_2, v_3) \text{ for } i=2$$

$$v_k.d \leq v_{k-1}.d + w(v_{k-1}, v_k) \text{ for } i=k$$

Summing up all, we get,
$$\sum_{i=1}^k v_i.d \leq \sum_{i=1}^k (v_{i-1}.d + w(v_{i-1}, v_i))$$

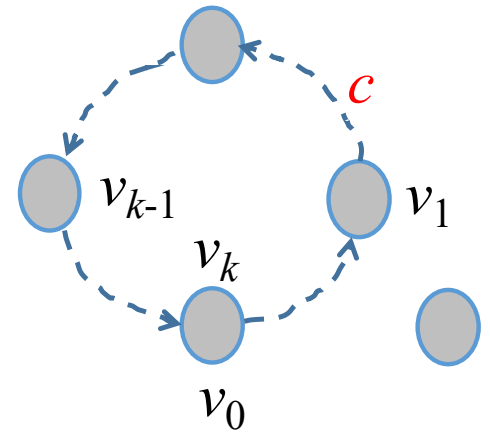


Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of B:

Assume for the contradiction, algorithm returns TRUE

$$\begin{aligned}\sum_{i=1}^k v_i \cdot d &\leq \sum_{i=1}^k (v_{i-1} \cdot d + w(v_{i-1}, v_i)) \\ &= \sum_{i=1}^k v_{i-1} \cdot d + \sum_{i=1}^k w(v_{i-1}, v_i)\end{aligned}$$



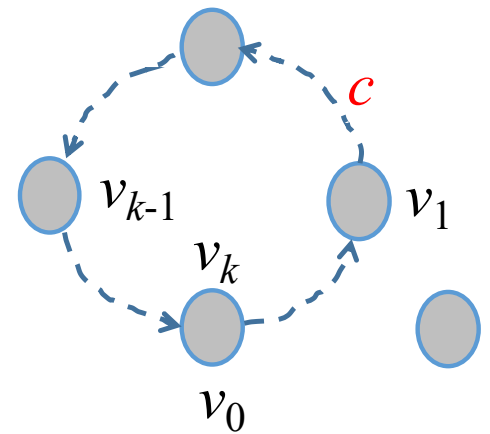
Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of B:

Assume for the contradiction, algorithm returns TRUE

$$\begin{aligned}\sum_{i=1}^k v_i \cdot d &\leq \sum_{i=1}^k (v_{i-1} \cdot d + w(v_{i-1}, v_i)) \\ &= \sum_{i=1}^k v_{i-1} \cdot d + \sum_{i=1}^k w(v_{i-1}, v_i)\end{aligned}$$

But, $v_0 = v_k$



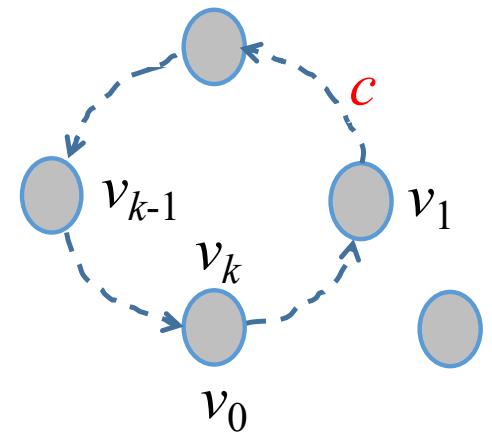
Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of B:

Assume for the contradiction, algorithm returns TRUE

$$\begin{aligned} \sum_{i=1}^k v_i \cdot d &\leq \sum_{i=1}^k (v_{i-1} \cdot d + w(v_{i-1}, v_i)) \\ &= \sum_{i=1}^k v_{i-1} \cdot d + \sum_{i=1}^k w(v_{i-1}, v_i) \end{aligned}$$

But, $v_0 = v_k$ therefore, $\sum_{i=1}^k v_i \cdot d = \sum_{i=1}^k v_{i-1} \cdot d$



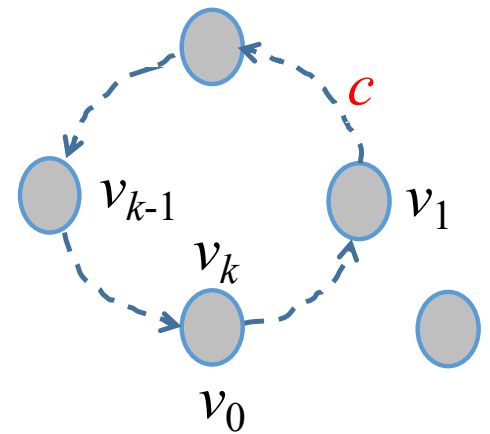
Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of B:

Assume for the contradiction, algorithm returns TRUE

$$\begin{aligned} \sum_{i=1}^k v_i \cdot d &\leq \sum_{i=1}^k (v_{i-1} \cdot d + w(v_{i-1}, v_i)) \\ &= \sum_{i=1}^k v_{i-1} \cdot d + \sum_{i=1}^k w(v_{i-1}, v_i) \end{aligned}$$

We get, $\sum_{i=1}^k w(v_{i-1}, v_i) \geq 0$



Bellman-Ford Algorithm Correctly Finds Shortest Paths

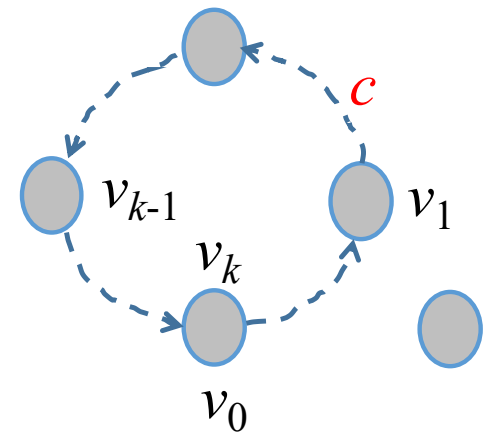
Proof of B:

Assume for the contradiction, algorithm returns TRUE

$$\begin{aligned} \sum_{i=1}^k v_i \cdot d &\leq \sum_{i=1}^k (v_{i-1} \cdot d + w(v_{i-1}, v_i)) \\ &= \sum_{i=1}^k v_{i-1} \cdot d + \sum_{i=1}^k w(v_{i-1}, v_i) \end{aligned}$$

We get, $\sum_{i=1}^k w(v_{i-1}, v_i) \geq 0$

Weight of c is non-negative!! Contradiction!!

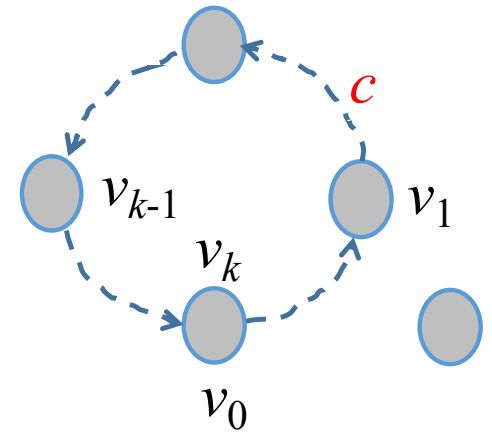
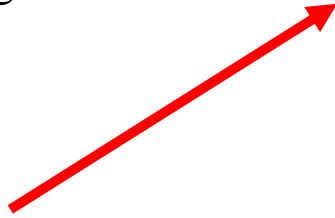


Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of B:

Assume for the contradiction, algorithm returns TRUE

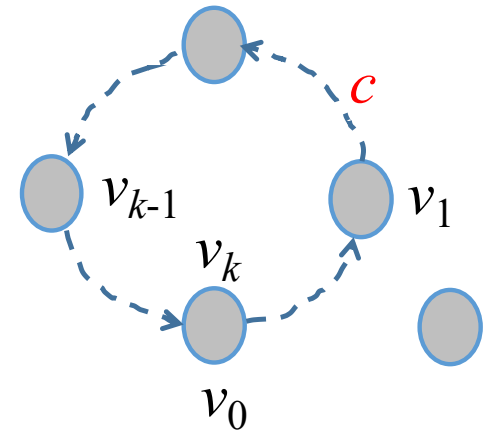
This assumption was wrong



Bellman-Ford Algorithm Correctly Finds Shortest Paths

Proof of B:

We proved that algorithm returns FALSE if negative-weight cycle exists.



Complexity of Bellman-Ford Algorithm

BELLMAN-FORD(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )   $O(V)$ 
2  for  $i = 1$  to  $|G.V| - 1$    $O(V)$ 
3      for each edge  $(u, v) \in G.E$    $O(E)$ 
4          RELAX( $u, v, w$ )   $O(1)$ 
5  for each edge  $(u, v) \in G.E$    $O(E)$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
```


Complexity of Bellman-Ford Algorithm

BELLMAN-FORD(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )   $O(V)$ 
2  for  $i = 1$  to  $|G.V| - 1$    $O(V)$ 
3      for each edge  $(u, v) \in G.E$    $O(E)$ 
4          RELAX( $u, v, w$ )   $O(1)$ 
5  for each edge  $(u, v) \in G.E$    $O(E)$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
```

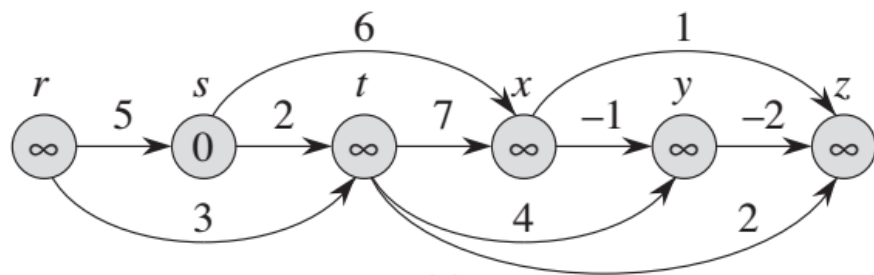
Total complexity $O(VE)$

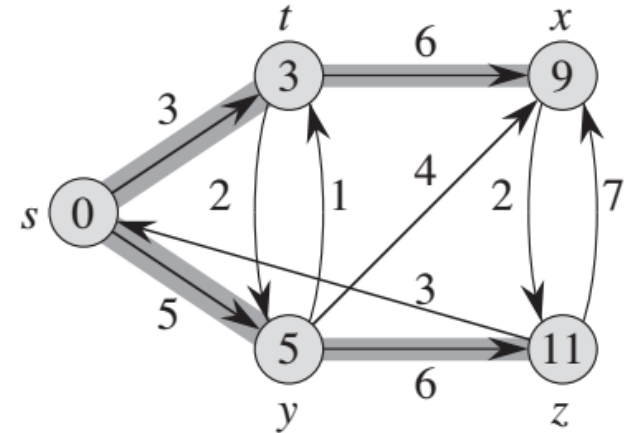
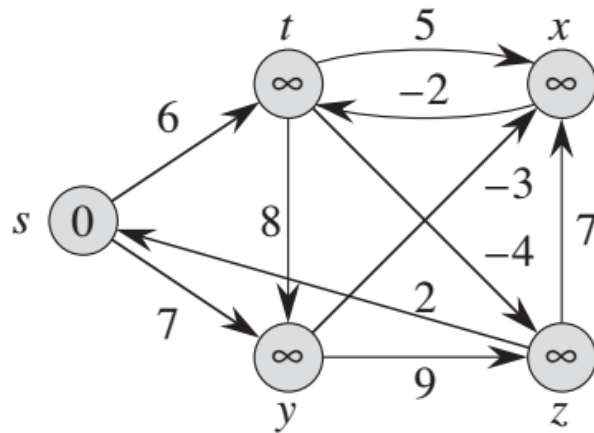
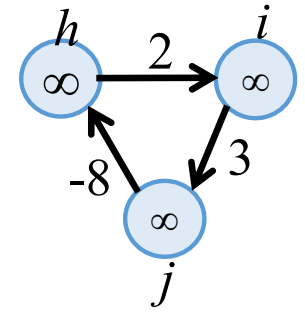
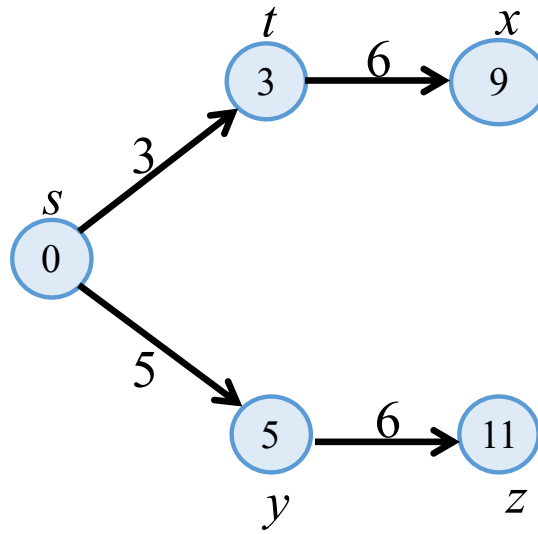
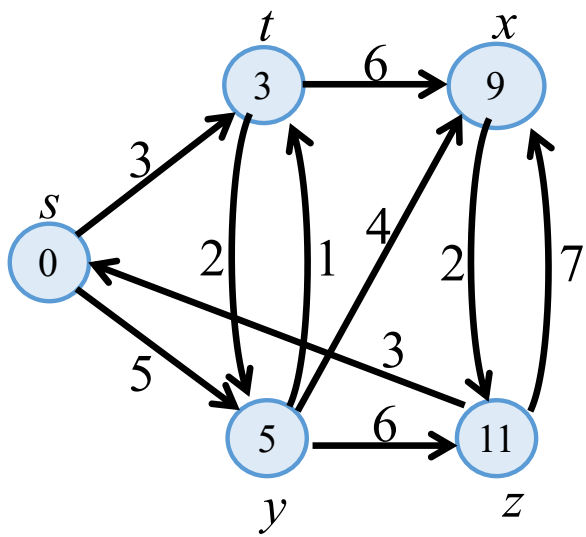
SSSP for DAG

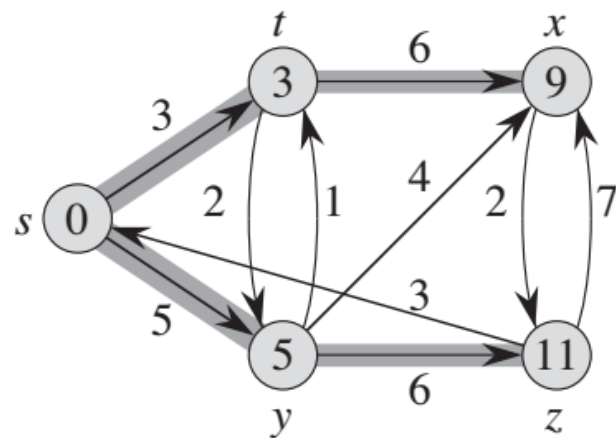
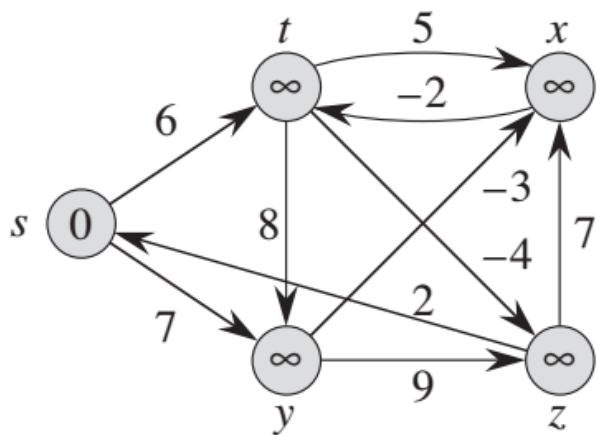
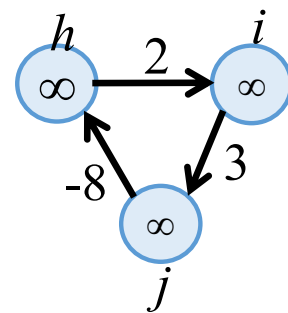
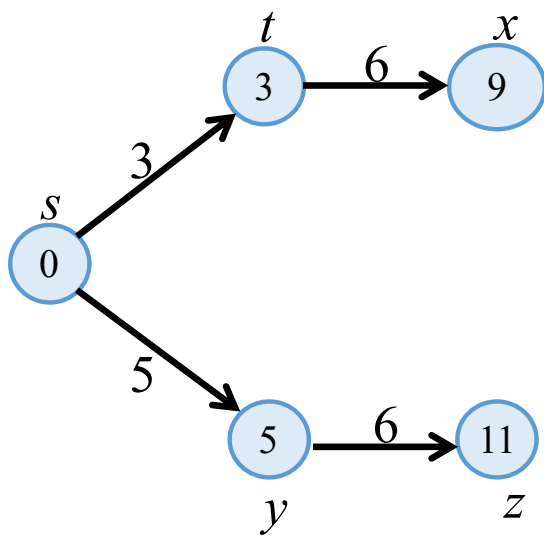
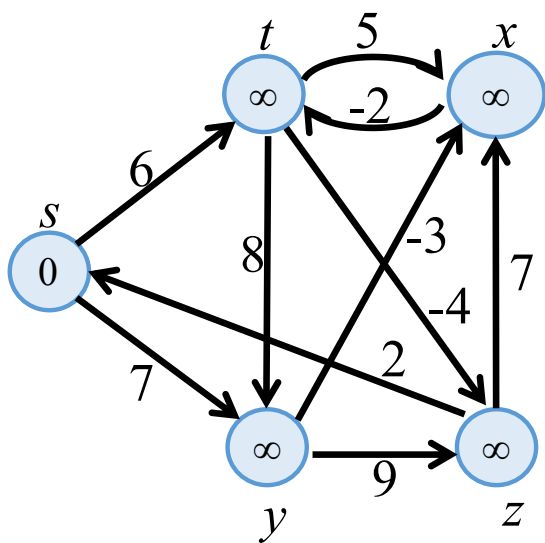
DAG-SHORTEST-PATHS(G, w, s)

- 1 topologically sort the vertices of G
- 2 INITIALIZE-SINGLE-SOURCE(G, s)
- 3 **for** each vertex u , taken in topologically sorted order
- 4 **for** each vertex $v \in G.Adj[u]$
- 5 RELAX(u, v, w)

Simulation of SSSP for DAG







End of Slides

Negative Weight Edges

